



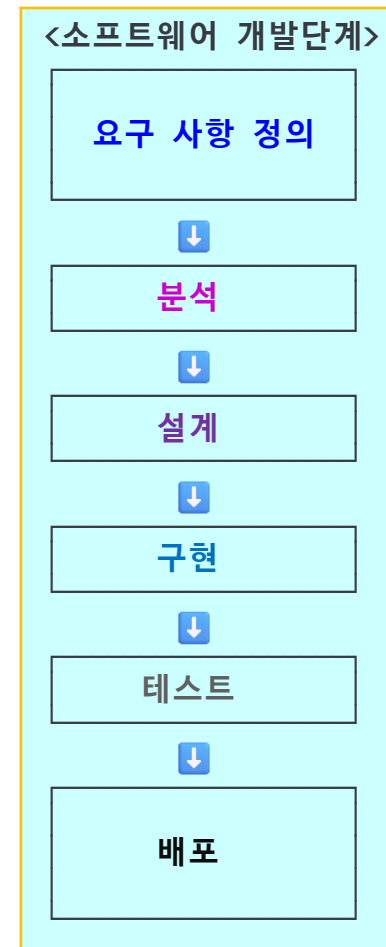
03. 클래스와 객체

Heesung Oh

<https://github.com/3dapi>



- 소프트웨어 모델링
- 절차적 구조와 객체지향 구조 비교
- 객체 모델에서 C++ 클래스로 전환
- class와 struct의 사용 기준
- 접근 제어와 캡슐화
- 멤버 함수와 this
- 정적 멤버와 클래스 상태
- 클래스 내부 선언과 friend
- 선언과 구현의 분리
- UML 다이어그램과 C++ 코드





3.1 소프트웨어 모델링

- 모델링: 요구 사항 분석, 프로그램으로 구현할 구조와 동작을 표현하는 과정
- 실제 시스템 전체를 그대로 옮기는 것이 아니라 필요한 요소를 선택하고 단순화
- 개발자 사이에서 구조를 공유하고 변경과 재작업을 줄이는 목적

산출물	표현 내용
요구 사항 명세서	필요한 기능과 조건
순서도	작업의 처리 순서와 제어 흐름
DFD	데이터의 입력, 처리, 이동
UML 다이어그램	객체의 구조, 관계, 협력

<소프트웨어 모델링>

요구 사항 정의



필요한 데이터와 기능 추출

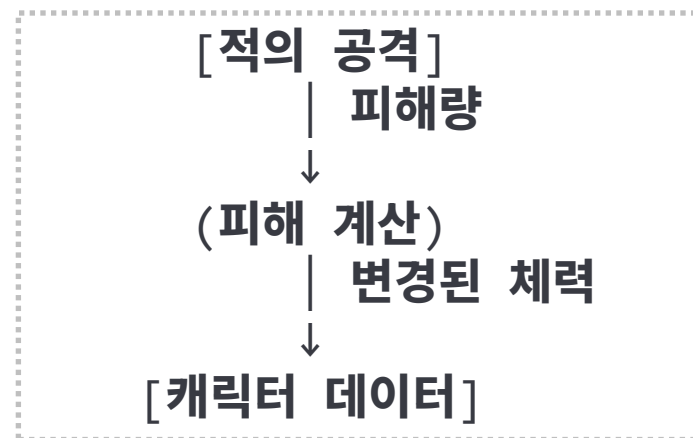
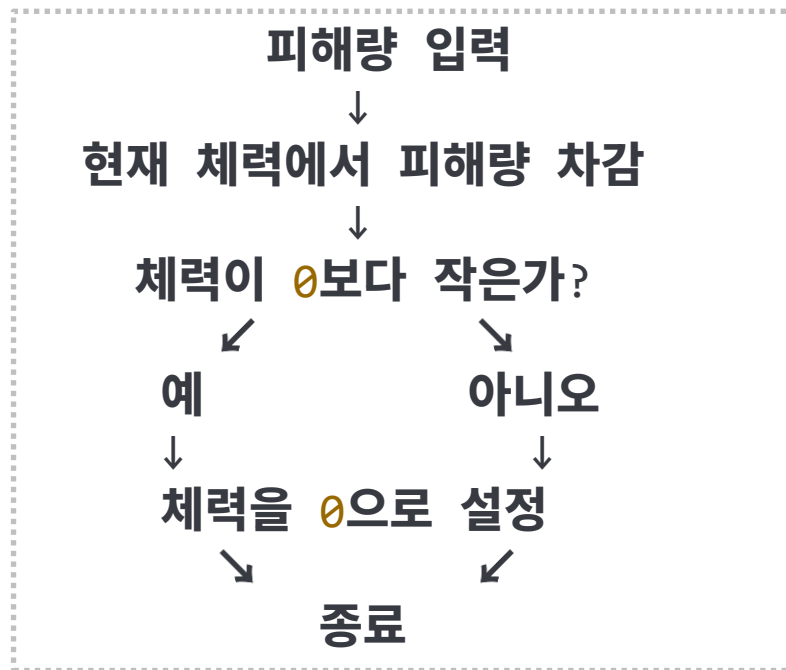


프로그램 구조로 표현





- 프로그램이 수행할 작업을 함수로 나눔
- 처리 순서와 데이터 흐름을 중심으로 구성
- 어떤 함수가 어떤 데이터를 처리하는지가 중요





```
struct Character
{
    int hp = 100;
};
void TakeDamage(Character* character, int damage)
{
    if (character->hp -= damage; character->hp < 0)
    {
        character->hp = 0;
    }
}
```

- **Character**는 데이터를 저장
- **TakeDamage()**는 전달받은 데이터를 변경
- 처리 흐름이 단순하면 함수 중심 구조도 명확함

```
void Move(Character* character);
void Attack(Character* character);
void TakeDamage(Character* character, int damage);
void Recover(Character* character, int amount);
```



- 프로그램이 커지면 여러 함수가 같은 데이터를 공유하고 변경
- 체력의 최소값, 최대값, 사망 조건 같은 규칙이 여러 함수에 흩어질 수 있음
- 규칙이 변경되면 관련된 함수를 모두 확인해야 함

Character 데이터

- TakeDamage()
- Recover()
- Load()
- Reset()
- Save()

핵심 문제

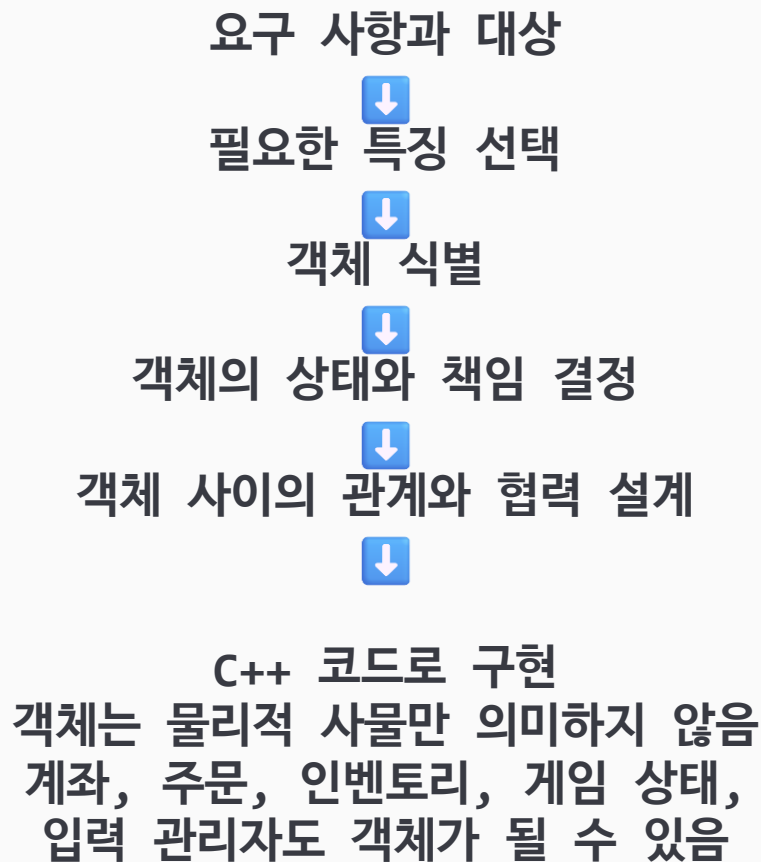
함수의 수가 많은 것 자체가 문제는 아님
상태와 변경 규칙을 누가 책임지는지가 중요
상태 + 변경 규칙



하나의 책임 단위로 묶는 구조 필요



- 프로그램에 필요한 대상을 객체로 나눔
- 각 객체가 자신의 상태와 책임을 관리
- 객체 사이의 관계와 협력을 중심으로 구조를 표현





● 객체지향 모델링의 질문

- ◆ 어떤 객체가 필요한가
- ◆ 객체가 어떤 상태를 가지는가
- ◆ 객체가 어떤 책임을 담당하는가
- ◆ 객체들이 어떤 관계를 맺는가
- ◆ 객체들이 어떻게 협력하는가
- ◆ 중요한 것은 데이터를 어느 객체가 관리하는지 결정
- ◆ 상태 변경 규칙은 그 상태를 가진 객체가 책임지는 구조

게임 캐릭터

└ 상태

├ 이름
├ 체력
├ 위치
└ 공격력

게임 캐릭터

└ 동작

├ 이동
├ 공격
└ 피해 처리



3.2.3 객체의 관계와 프로그램 구조

- 객체는 독립된 부품처럼 구성할 수 있음
- 작은 객체를 연결하여 더 큰 객체와 프로그램으로 확장
- 객체 사이의 관계에서 새로운 객체가 도출될 수 있음

학생

- 학번
- 이름

학생 <— 강좌 —> 과목

과목

- 과목 코드
- 과목명
- 이수 학점

강좌

- 과목
- 수강 학생 목록
- 담당 교수
- 강의실
- 수업 시간
- 개설 학기



3.2.4 객체의 협력과 실행 흐름

- 객체지향 모델링은 구조뿐 아니라 객체들이 어떤 순서로 요청하는지도 설계
- Player가 모든 데이터를 직접 변경하면 책임이 커짐
- 각 객체가 자신의 상태와 기능을 담당하도록 구성

Player —공격력 요청—▶ Weapon
Weapon —공격력 반환—▶ Player
Player —피해 처리 요청—▶ Enemy
Enemy —체력 변경—▶ 자신의 상태

객체	책임
Player	공격 시작, 사용할 무기 선택
Weapon	공격력 관리와 제공
Enemy	체력 관리와 피해 처리



3.2.5 절차적 구조와 객체지향 구조 비교

- 객체지향 프로그래밍은 절차적 코드를 없애는 방식이 아님
- 핵심은 책임의 배치

구분	절차적 구조	객체지향 구조
중심	처리 순서와 함수	객체의 상태와 책임
데이터	여러 함수가 공유	객체가 관리
규칙	함수마다 알고 있을 수 있음	객체 내부로 모음
적합한 상황	단순 계산, 순서 중심 처리	상태와 규칙이 많은 대상

절차적 구조

데이터 → 함수 → 함수 → 함수

객체지향 구조

객체 —메시지—▶ 객체 —메시지—▶ 객체



- UML은 객체지향 설계 내용을 다이어그램으로 표현하기 위한 모델링 언어
- 프로그램을 실행하는 언어가 아님
- 자동으로 코드를 완성해 주는 방법도 아님
- 구현 전에 구조를 검토하고 개발자 사이에서 공유하는 표현 수단

다이어그램	표현 내용
클래스 다이어그램	객체의 상태, 기능, 클래스 사이의 관계
시퀀스 다이어그램	객체의 요청과 처리 순서

객체의 상태와 기능, 관계 → 클래스 다이어그램
객체의 요청과 처리 순서 → 시퀀스 다이어그램





3.3 객체 모델에서 C++ 클래스로

- 객체지향 모델링에서 찾은 객체는 구현해야 할 논리적 대상
- 객체의 상태는 데이터로 표현
- 객체의 기능은 함수로 표현
- C++에서는 데이터와 기능을 하나의 **사용자 정의 타입**으로 묶기 위해 **클래스**를 사용

- 객체의 상태 → 멤버 변수
- 객체의 기능 → 멤버 함수
- 객체의 상태, 기능 정의 → 클래스

```
class Character
{
public:
    void Move(int x, int y);
    void Attack();
    void TakeDamage(int damage);

private:
    int hp = 100;
    int x = 0, y = 0;
    int attackPower = 10;
};
```



3.3.2 멤버 변수와 멤버 함수

- 클래스 안에 선언한 변수는 객체의 상태를 저장
- 클래스 안에 선언한 함수는 객체가 수행할 기능과 상태 변경 규칙을 표현
- 멤버 함수는 같은 객체의 멤버 변수에 직접 접근 가능

```
class Character
{
public:
    void TakeDamage(int damage);

private:
    int hp = 100;
};

void Character::TakeDamage(int damage)
{
    if (hp -= damage; hp < 0)
    {
        hp = 0;
    }
}
```

- 외부에서는 hp를 직접 변경하지 않고 TakeDamage()를 호출
- 체력 변경 규칙은 Character가 관리



- 클래스 자체는 타입 정의
- 클래스 타입으로 변수를 선언하면 실제 객체 생성
- 같은 클래스에서 생성된 객체라도 각자 자신의 멤버 변수를 가짐

Character player;
Character enemy;

Character 클래스



용어	의미
클래스	객체의 상태와 기능을 정의한 C++ 사용자 정의 타입
객체	클래스에 따라 생성되어 저장 공간을 가지는 실체
인스턴스	특정 클래스에 따라 생성된 객체임을 강조하는 표현



3.4 객체지향 프로그래밍의 주요 특징

특징	의미
추상화	필요한 상태와 기능만 선택하여 모델로 표현
캡슐화	상태와 기능을 묶고 내부 상태 변경을 제한
상속	기존 클래스의 특성을 바탕으로 새 클래스 정의
다형성	같은 인터페이스로 서로 다른 객체가 각자의 방식으로 동작

```
class Character
{
public:
    void TakeDamage(int damage)
    {
        if (hp -= damage; hp < 0)
        {
            hp = 0;
        }
    }

private:
    int hp = 100;
};
```

- 클래스는 네 가지 특징을 자동으로 완성하는 문법이 아님
- 접근 제어, 상속, 가상 함수 등을 함께 사용하여 객체지향 구조를 구현



- C++의 class와 struct는 모두 **클래스 타입**을 정의
- 사용할 수 있는 기능은 거의 같음
- 기본 접근 수준과 표현하는 **설계 의도**가 다름

구분	class	struct
기본 멤버 접근 수준	private	public
기본 상속 접근 수준	private	public
멤버 함수	가능	가능
생성자와 소멸자	가능	가능
상속과 가상 함수	가능	가능

```
class ClassTypeSample
{
    int value; // private
};

struct StructTypeSample
{
    int value; // public
};
```



- 서로 관련된 여러 데이터를 하나로 묶는 것이 목적이면 struct가 자연스러움
- 외부에서 멤버를 직접 읽고 변경해도 타입의 의미가 손상되지 않는 경우에 적합
- 구조체로 표현하기 좋은 데이터
 - ◆ 좌표와 크기
 - ◆ 색상값
 - ◆ 날짜와 시간
 - ◆ 정점 데이터
 - ◆ 파일이나 네트워크에서 읽은 단순 데이터
 - ◆ 함수의 여러 반환값

```
struct Position  
{  
    int x, y;  
};
```

```
Position playerPosition;  
playerPosition.x = 100;  
playerPosition.y = 200;
```



3.5.4 값 의미를 가지는 struct

- 값 자체가 중요한 타입은 struct로 표현하기 좋음
- 복사된 값은 원본과 독립적으로 사용
- 멤버를 직접 읽고 변경해도 의미가 손상되지 않는 경우에 적합

```
struct Position
{
    int x, y;

    void Move(int offsetX, int offsetY)
    {
        x += offsetX;
        y += offsetY;
    }
};
```

```
Position first{ 10, 20 };
Position second = first;
second.x = 50;
```

```
first  → (10, 20)
second → (50, 20)
```



3.5.5 객체 설계에 사용하는 class

- 객체가 자신의 상태와 기능을 관리해야 한다면 class가 자연스러움
- 상태 변경 규칙이 있는 데이터는 외부에서 직접 변경하지 못하게 구성

```
class BankAccount
{
public:
    void Deposit(int amount)
    {
        // ...
    }
    bool Withdraw(int amount)
    {
        // ...
        return true;
    }
    int GetBalance() const
    {
        // ...
    }
private:
    int balance = 0;
};
```



3.5.6 상태와 규칙이 있는 객체

- 객체의 상태에는 반드시 지켜야 하는 규칙이 필요
- 객체가 항상 만족해야 하는 상태 조건을 불변식
- 데이터를 은닉보다 상태 변경 규칙을 객체가 책임

BankAccount 규칙

- 입금액은 0보다 커야 함
- 출금액은 0보다 커야 함
- 잔액보다 많은 금액은 출금 불가
- 잔액은 0보다 작아질 수 없음

Withdraw() 호출



출금액과 잔액 확인



가능



잔액 감소



불가능



상태 유지





- 객체를 설계할 때의 기본 선택 → class
- 단순한 데이터나 값의 표현 → struct

구분	class	struct
기본 접근 수준	private	public
주된 용도	상태와 기능을 관리하는 객체	단순 데이터와 값 표현
멤버 접근	공개된 멤버 함수를 통해 접근	직접 접근하는 경우가 많음
상태 변경 규칙	객체가 직접 관리	없거나 단순함
대표 예	BankAccount, Inventory, Player	Position, Vector2, Color

- 이 기준은 절대적인 규칙이 아님
- 타입의 역할과 사용 의도가 코드에서 분명하게 드러나는가를 기준으로 선택



3.6 접근 제어와 캡슐화

- 접근 제어는 클래스 멤버를 어디에서 사용할 수 있는지 제한
- public, protected, private 접근 지정자 사용
- 캡슐화는 상태와 기능을 객체 단위로 묶고 외부 접근을 제어

접근지정자	클래스내부	파생클래스	클래스외부
private	가능	불가능	불가능
protected	가능	가능	불가능
public	가능	가능	가능

```
class Player
{
    public:
        void Move();

    protected:
        int level = 1;

    private:
        int hp = 100;
};
```



3.6.1 public

- public 멤버는 클래스 외부에서 접근 가능
- 외부 객체가 사용할 수 있도록 공개한 멤버 함수는 공개 인터페이스 구성
- 객체를 사용하는 데 필요한 기능만 외부에 공개

```
class Door
{
public:
    void Open()
    {
        isOpen = true;
    }

    void Close()
    {
        isOpen = false;
    }
}
```

```
    bool IsOpen() const
    {
        return isOpen;
    }

private:
    bool isOpen = false;
};
```

```
Door door;
door.Open();
door.Close();
```




3.6.2 protected와 private

- Protected → 클래스 외부에서는 접근 불가
- 해당 클래스를 상속한 파생 클래스에서는 접근 가능
- Private → 클래스 내부의 멤버 함수에서만 직접 접근 가능

```
class Character
{
protected:
    int hp = 100;
};

class Player : public Character
{
public:
    void Recover()
    {
        hp += 10;
    }
};
```

```
class Counter
{
public:
    void Increase()
    {
        ++count;
    }

private:
    int count = 0;
};
```



- 객체 사용에 필요하지 않은 내부 구현을 외부에 노출안함
- 외부에서는 객체가 제공하는 기능만 사용
- 내부 저장 방식이 바뀌어도 공개 인터페이스가 유지되면 외부 코드 영향 감소

외부 코드

- AddItem() 요청
- RemoveItem() 요청
- GetItemCount() 요청



Inventory
내부 저장 방식
내부 변경 규칙

```
class Inventory
{
public:
    bool AddItem(int itemId);
    bool RemoveItem(int itemId);
    int GetItemCount() const;

private:
    int itemIds[100];
    int itemCount = 0;
};
```



- 불변식 유지
- 값이 변경된 뒤에도 불변식 유지
- 외부에서 상태를 직접 변경하면 잘못된 상태가 만들어질 수 있음

```
struct HealthData  
{  
    int current, maximum;  
};
```

```
HealthData health;  
health.current = -100;  
health.maximum = 0;
```

문법적으로 가능하지만 체력 데이터로는 올바르지 않음



● 불변식 우회하는 포인터 연산은 UB 유발

```
#include <iostream>
using namespace std;

class PlayerStats
{
private:
    // 외부의 직접 변경 제한.
    // 클래스의 변경 규칙을 통한
    // 불변식(Invariant) 유지.
    int hp{10}, mp{20}, level{30};

public:
    PlayerStats() = default;
    int GetHp() const { return hp; }
    int GetMp() const { return mp; }
    int GetLevel() const { return level; }
};
```

```
int main()
{
    PlayerStats playerStat;
    for(int i = 0; i < 3; ++i)
    {
        // 위험: 접근 제어와 캡슐화 우회
        // 객체 시작 주소를 int*로 강제 변환.
        // 포인터 연산으로 객체 내부 메모리에 직접 접근.
        int* data = (int*)&playerStat + i;

        // 위험: 객체의 불변식 파괴. private 접근 제어 우회
        // 상태 변경 규칙을 거치지 않고 값 변경.
        // 객체를 int 배열처럼 가정한 접근
        // Undefined Behavior 발생 가능.
        *data = 100 + i * i + i * 500;
    }
    // 강제 메모리 변경 이후의 객체 상태 확인
    cout << "HP  :" << playerStat.GetHp() << '\n';
    cout << "MP  :" << playerStat.GetMp() << '\n';
    cout << "Level:" << playerStat.GetLevel() << '\n';
}
```



3.6.6 접근자 함수 사용 시 주의점

- 비공개 멤버마다 기계적으로 게터와 세터를 만들면 캡슐화가 보장되지 않음
- 모든 값을 그대로 읽고 변경할 수 있으면 공개 멤버 변수와 큰 차이가 없을 수 있음
- 상태 변경 규칙이 있다면 의미가 분명한 멤버 함수를 제공

```
class Player
{
public:
    void SetHp(int value)
    {
        hp = value;
    }

private:
    int hp = 100;
};
```

```
Player player;
player.SetHp(-1000);
```

SetHp(-1000) → 외부에서 객체 상태를 임의로 결정
TakeDamage(30) → 객체가 피해 처리 규칙에 따라 상태 변경



3.6.6 비상수 참조 반환 주의

- 비공개 멤버의 비상수 참조를 반환하면 외부 코드가 내부 상태를 직접 변경 가능
- 접근 제어를 우회하여 내부 상태를 공개하는 것과 비슷한 결과가 될 수 있음

```
#include <string>
class Profile
{
public:
    std::string& GetName()
    {
        return name;
    }
private:
    std::string name;
};
```

```
Profile profile;
profile.GetName() = "Player";
```

- 읽기만 필요하면 **const** 참조 반환을 고려
- 상태 변경에는 의미가 분명한 함수 제공

```
const std::string& GetName() const;
bool Rename(const std::string& newName);
```



3.7 멤버 함수와 this

- 멤버 함수는 객체의 상태를 처리하거나 객체가 제공할 기능을 구현
- 일반 함수는 필요한 데이터를 매개변수로 전달받음
- 멤버 함수는 자신을 호출한 객체의 멤버에 직접 접근

```
struct Vector2 { float x, y; };  
class Player  
{  
public:  
    void Move(float offsetX, float offsetY)  
    {  
        position.x += offsetX;  
        position.y += offsetY;  
    }  
private:  
    Vector2 position = { 0.0f, 0.0f };  
};  
  
Player first;  
Player second;  
first.Move(10.0f, 20.0f);  
second.Move(-5.0f, 30.0f);
```



- this는 현재 멤버 함수를 호출한 객체를 가리키는 포인터
- 정적 멤버 함수가 아닌 모든 멤버 함수 안에서 사용 가능
- 매개변수와 멤버 이름이 같을 때 구분에 사용

```
class Window
{
public:
    void SetSize(int width, int height)
    {
        this->width = width;
        this->height = height;
    }
private:
    int width = 800, height = 600;
};
```

표현	의미
this	현재 객체의 주소
*this	현재 객체
this->width	현재 객체의 멤버 width
(*this).width	현재 객체의 멤버 width



- 현재 객체를 가리키는 포인터는 this
- *this는 현재 객체 자체를 의미
- 같은 객체를 계속 사용하려면 일반적으로 참조 타입 반환

```
class RenderOption
{
public:
    RenderOption& SetFullScreen(bool enabled)
    {
        fullScreen = enabled;
        return *this;
    }
private:
    bool fullScreen = false
}
```

반환 타입	반환 표현	의미
RenderOption	return *this;	현재 객체의 복사본 반환
RenderOption&	return *this;	현재 객체 참조 반환
RenderOption*	return this;	현재 객체 주소 반환



- 객체 자신을 참조로 반환하면 멤버 함수를 연속해서 호출 가능
- 여러 단계로 객체를 설정하는 코드를 간결하게 만들 수 있음

```
RenderOption option;  
option.SetResolution(1920, 1080)  
    .SetFullScreen(true)  
    .SetVSync(false);
```

```
option.SetResolution(1920, 1080)
```

└ option의 참조 반환



```
.SetFullScreen(true)
```



```
.SetVSync(false)
```

- 체이닝을 위해 성공 여부 확인이 필요한 함수를 억지로 참조 반환으로 바꾸지 않음



- 객체의 상태를 변경하지 않는 멤버 함수는 매개변수 목록 뒤에 `const`를 붙임
- 상수 멤버 함수에서는 일반 멤버 변수를 변경할 수 없음
- 상수 객체는 상수 멤버 함수만 호출 가능

```
class RenderOption
{
public:
    int GetWidth() const
    {
        return width;
    }

private:
    int width = 1280;
};
```

멤버 함수	의미
int GetWidth() const	객체 상태를 변경하지 않음
void SetWidth(int)	객체 상태 변경 가능

- **const**는 호출 객체의 상태를 변경하지 않는다는 의도를 인터페이스에 표현



3.8 정적 멤버

- 비정적 데이터 멤버는 객체마다 별도의 저장 공간을 가짐
- 정적 데이터 멤버는 클래스 전체에서 하나만 존재
- 모든 객체가 공유하는 클래스 상태를 표현

구분	비정적 데이터 멤버	정적 데이터 멤버
소속	각각의 객체	클래스
저장 공간	객체마다 별도	클래스 전체에서 하나
값	객체마다 다를 수 있음	모든 객체가 공유
객체생성 필요	객체가 생성되어야 존재	객체가 없어도 사용 가능
접근 방법	객체를 통해 접근	클래스 범위로 접근

```
class Player
{
public:
    int hp = 100;
    static int maxPlayerCount;
};

int Player::maxPlayerCount = 4;
```



- 클래스 안에서 선언하지만 일반적으로 클래스 밖에서 정의
- 클래스 밖 정의에는 `static`을 다시 쓰지 않음
- 정적 데이터 멤버는 객체의 저장 공간에 포함되지 않음

```
class Player
{
public:
    int hp = 100;
    static int maxPlayerCount;
};

int Player::maxPlayerCount = 4;

std::cout << Player::maxPlayerCount << '\n';
Player::maxPlayerCount = 8;
```

Player 클래스
└─ maxPlayerCount = 8

first 객체
└─ hp = 100

second 객체
└─ hp = 100



- 정적 멤버 함수는 특정 객체가 아니라 클래스에 속한 함수
- 객체를 생성하지 않고 클래스 범위로 호출 가능
- this 포인터가 없음 → 클래스는 스코프
- 비정적 데이터 멤버에 직접 접근할 수 없음

```
class Player
{
public:
    static void SetMaxPlayerCount(int count)
    {
        if (count > 0)
        {
            maxPlayerCount = count;
        }
    }
    static int GetMaxPlayerCount()
    {
        return maxPlayerCount;
    }
private:
    inline static int maxPlayerCount = 4;
};
```

```
Player::SetMaxPlayerCount(8);
std::cout << Player::GetMaxPlayerCount() << '\n';
```



3.8.3 인라인 정적 멤버

- C++17부터 정적 데이터 멤버에 inline을 사용하면 클래스 안에서 정의와 초기화를 함께 작성 가능
- 클래스 밖에서 별도의 정의를 작성하지 않아도 됨
- constexpr 정적 데이터 멤버는 C++17부터 암시적으로 inline

```
class GameConfig
{
public:
    inline static int maxPlayerCount = 4;
    inline static const int minimumPlayerCount = 1;
    static constexpr int maximumStageCount = 100;
```

선언	변경 가능 여부	컴파일 타임 상수
inline static int	가능	아님
inline static const int	불가능	초기화 방식에 따라 다름
static constexpr int	불가능	가능



3.8.4 객체 상태와 클래스 상태

- 객체마다 서로 다른 값을 가져야 하면 비정적 데이터 멤버
- 모든 객체가 공유해야 하면 정적 데이터 멤버
- 값의 의미를 기준으로 판단

```
class Player
{
private:
    Vector2 position = { 0.0f, 0.0f };
    int hp = 100, level = 1;
    inline static float movementLimit = 1000.0f;
};
```

멤버	상태 종류	저장 개수
position	객체 상태	객체마다 하나
hp	객체 상태	객체마다 하나
level	객체 상태	객체마다 하나
movementLimit	클래스 상태	클래스 전체에서 하나



3.8.5 전역 변수와 정적 멤버

- 전역 변수와 정적 데이터 멤버는 모두 프로그램 실행 동안 하나의 저장 공간 유지
- 정적 데이터 멤버는 특정 클래스에 속한다는 의미를 표현
- 접근 제어와 정적 멤버 함수로 변경 규칙을 적용 가능

```
class Player
{
public:
    static bool SetMaxPlayerCount(int count)
    {
        if (count < 1 || count > 100)
        {
            return false;
        }

        maxPlayerCount = count;
        return true;
    }

private:
    inline static int maxPlayerCount = 4;
}
```

구분	전역 변수	정적 데이터 멤버
소속	특정 클래스에 속하지 않음	클래스에 속함
접근 제어	접근 지정자 사용 불가	접근 지정자 사용 가능
의미 전달	소속이 불분명할 수 있음	어떤 타입의 상태인지 명확함



3.9 클래스 내부(Nested)의 선언

- 클래스 내부에는 클래스, 열거형, 타입 별칭 같은 새로운 타입 선언 가능
- 클래스 내부에 선언된 이름은 해당 클래스의 범위에 속함
- 외부에서는 클래스 **범위 연산자 ::**를 사용

```
class Inventory
{
public:
    class Item;
    enum class State;
    using ItemId = int;
};
```

```
Inventory::Item
Inventory::State
Inventory::ItemId
```

장점	설명
이름의 소속 표현	어떤 클래스와 관련되는지 명확함
이름 충돌 방지	클래스 범위 안에 이름 제한
관련 선언 구성	함께 쓰는 타입을 클래스 안에 모음



3.9.1 중첩 클래스

- 중첩 클래스 → 클래스 내부에 선언된 클래스
- 중첩은 이름의 소속만 나타낼 뿐 자동 포함 관계를 만들지 않음
- 접근 수준에 따라 외부에서 타입 이름을 사용할 수 있는지 결정

```
class Inventory
{
public:
    class Item
    {
    public:
        void Set(int itemId, int itemCount) {
            id = itemId;
            count = itemCount;
        }
        int GetId() const { return id; }
    private:
        int id = 0, count = 0;
    };
private:
    Item items[100];
    int itemCount = 0;
};
```

```
Inventory::Item item;
item.Set(1001, 5);
```



3.9.2 중첩 열거형과 타입 별칭

- 클래스가 사용하는 상태나 종류를 클래스 내부 열거형으로 선언 가능
- 특정 클래스에서만 사용 → 내부 선언이 의미를 명시
- 타입 별칭은 클래스가 사용하는 타입의 역할을 표현

```
class Character
{
public:
    enum class State
    {
        Idle,
        Move,
        Attack,
        Dead
    };
    void SetState(State newState)
    {
        state = newState;
    }
private:
    State state = State::Idle;
};
```

```
class Inventory
{
public:
    using ItemId = int;
    using ItemCount = int;

    bool AddItem(ItemId id, ItemCount count);
};
```



- friend 함수는 클래스의 멤버 함수가 아님
- 해당 클래스의 private, protected 멤버 접근을 허용받은 일반 함수
- 왼쪽 피연산자가 클래스 객체가 아닌 연산자를 정의할 때 사용 가능

```
#include <ostream>
class Vector2
{
public:
    Vector2(float x, float y) : x(x), y(y) {}

    friend std::ostream& operator<<(std::ostream& output, const Vector2& value);

private:
    float x, y;
};
```

friend 선언은 접근 권한을 부여할 뿐 멤버 함수로 바꾸지 않음



3.9.5 friend 클래스와 사용 주의

- 특정 클래스의 모든 멤버 함수에 비공개 멤버 접근을 허용하려면 friend class 사용
- 친구 관계는 단방향, 비대칭, 비상속, 비전이
- 내부 구현 의존성을 높일 수 있으므로 제한적으로 사용

```
class GameData
{
    friend class SaveSystem;

private:
    int score = 0, stage = 1;
};

class SaveSystem
{
public:
    static void Save(const GameData& data)
    {
        std::cout << "Score: " << data.score << '\n';
        std::cout << "Stage: " << data.stage << '\n';
    }
}
```

필요한 접근	권장 방법
하나의 함수만 비공개 멤버 접근	해당 함수만 friend
값을 읽는 기능만 필요	공개 멤버 함수 제공
단순한 편의	friend 사용 재검토



- 작은 예제는 클래스 정의와 멤버 함수 본문을 한 파일에 작성 가능
- 프로젝트에서는 선언과 구현을 헤더 파일과 소스 파일로 분리
- 헤더 파일 → 클래스가 제공하는 기능 확인
- 소스 파일 → 각 기능의 처리 방법 구현

Player.h

└─ Player 클래스의 구성과 멤버 함수 선언

Player.cpp

└─ Player 멤버 함수의 실제 처

파일	주요 내용
헤더 파일 .h, .hpp	클래스와 구조체 정의, 함수 선언, 열거형, 상수
소스 파일 .cpp	함수와 멤버 함수 정의
프로그램 시작 파일 .cpp	main() 함수와 객체 사용 코드



● 헤더 파일

- ◆ 헤더 파일이 여러 번 포함되면 같은 타입이 중복 정의될 수 있음
- ◆ `#pragma once`는 하나의 번역 단위에서 헤더가 한 번만 포함되도록 함
- ◆ 전통적인 헤더 가드 `#ifndef ~ #endif` 도 같은 목적

```
// Vector2.h
#pragma once

struct Vector2
{
    float x = 0.0f, y = 0.0f
}
```

```
// Vector2.h
#ifndef VECTOR2_H
#define VECTOR2_H

struct Vector2
{
    ...
}

#endif
```

포함 방식	일반적인 용도
<code>#include <string></code>	표준 라이브러리나 외부 라이브러리
<code>#include "Vector2.h"</code>	현재 프로젝트에서 작성한 헤더



```
// Player.h
#pragma once

#include <string>
#include "Vector2.h"

class Weapon;

class Player
{
public:
    Player(const std::string& name);
    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);
    void EquipWeapon(Weapon* newWeapon);
    int GetHp() const;
    int GetAttackPower() const;
    const std::string& GetName() const;
    const Vector2& GetPosition() const;

private:
    std::string name;
    Vector2 position;
    Weapon* weapon = nullptr;
    int hp = 100;
};
```

- 멤버 함수의 이름, 매개변수, 반환 타입만 작성하고 본문은 생략
- 컴파일러는 선언을 통해 함수를 어떻게 호출해야 하는지 알 수 있음



```
// Player.cpp
#include "Player.h"
#include "Weapon.h"

Player::Player(const std::string& name)
    : name(name)
{
}

void Player::Move(float offsetX
                  , float offsetY)
{
    position.x += offsetX;
    position.y += offsetY;
}

void Player::TakeDamage(int damage)
{
    if (damage <= 0)
    {
        return;
    }

    if (hp -= damage; hp < 0)
    {
        hp = 0;
    }
}
```

- 클래스 밖에서 멤버 함수를 정의할 때는 **Player::Move**처럼 소속 지정
- 헤더의 선언과 소스 파일의 정의는 이름, 반환 타입, 매개변수, **const**가 일치해야 함



- 인터페이스는 외부에서 사용할 수 있도록 공개한 기능의 집합
- 구현은 그 기능이 내부에서 실제로 처리되는 방법
- 구현이 바뀌어도 인터페이스가 유지되면
외부 호출 방법은 바뀌지 않음

// 공개 인터페이스

class Player

{

public:

void Move(float offsetX, float offsetY);

void TakeDamage(int damage);

int GetHp() **const**;

};

// 구현은 바뀔 수 있음

void Player::Move(float offsetX, float offsetY)

{

position.x += offsetX;

position.y += offsetY;

}

player.Move(10.0f, 20.0f); // 호출 방법 유지

player.Move(10.0f, 20.0f); // 호출 방법 유지



- 전방 선언은 타입의 이름만 컴파일러에 알리는 선언
- 전체 정의가 알려지지 않은 타입은 불완전 타입
- 포인터와 참조 선언에는 전체 크기가 필요하지 않음
- 값 타입 멤버, 객체 생성, 멤버 함수 호출에는 전체 정의 필요

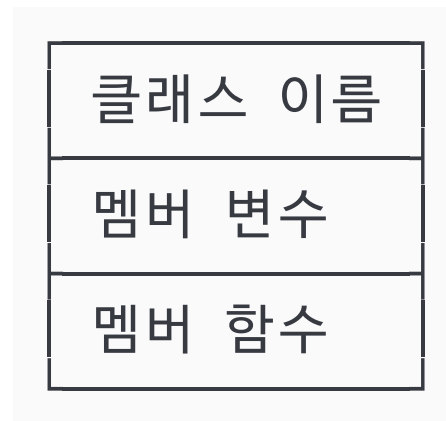
```
class Weapon;  
  
class Player  
{  
public:  
    void EquipWeapon(Weapon* newWeapon);  
  
private:  
    Weapon* weapon = nullptr;  
};
```

작업	전방 선언만으로 가능
포인터 선언	가능
참조 선언	가능
포인터나 참조를 사용하는 함수 선언	가능
값 타입 데이터 멤버 선언	불가능
객체 생성	불가능
멤버 함수 호출	불가능



- UML은 C++ 문법을 그대로 옮겨 적는 표기법이 아님
- 클래스, 상태, 기능, 관계와 동작 순서를 코드보다 간결하게 표현
- 클래스 다이어그램은 정적인 구조
- 시퀀스 다이어그램은 시간에 따른 협력

다이어그램	표현하는 내용
클래스 다이어그램	클래스의 데이터와 기능, 클래스 사이의 관계
시퀀스 다이어그램	객체가 시간의 흐름에 따라 주고받는 메시지





3.11.1 클래스 다이어그램

Character
- name : std::string - position : Vector2 - hp : int = 100
+ Move(x : float , y : float) : void + TakeDamage(damage : int) : void + GetName() : const std::string& + GetPosition() : const Vector2& + GetHp() : int

UML 표기	C++ 코드
- hp : int = 100	private: int hp = 100;
+ Move(...) : void	public: void Move(...);
+ GetHp() : int	public: int GetHp() const;

기호	접근 수준
+	public
-	private
#	protected





3.11.2 클래스 사이의 관계

관계	의미	C++에서 주로 나타나는 형태
일반화	상위 타입의 특성을 하위 타입이 이어받음	상속
의존	다른 객체를 일시적으로 사용	매개변수, 반환값, 지역 변수
연관	다른 객체와 지속적으로 연결	포인터나 참조 멤버
집합	독립 객체를 모아 관리	외부 객체의 포인터나 참조 저장
합성	구성 요소를 소유	값 타입 멤버 또는 독점 소유

- UML 관계가 특정 C++ 문법 하나와 항상 정확히 일치하는 것은 아님
- 객체의 역할, 소유권, 수명까지 함께 판단





- 일반화는 is-a 관계
 - ◆ Player는 Character 이다
 - ◆ Enemy는 Character 이다
- 의존은 함수 실행 중 일시적으로 사용하는 관계

```
Player —————> Character
Enemy  —————> Character
```

```
class Player : public Character
{
public:
    void Attack(Weapon& weapon, Enemy& enemy);
}

class Enemy : public Character
{
public:
    void Damaged(Player& player);
}
```




관계	핵심 질문
연관: Association	지속적으로 알고 있는가
집합: Aggregation	부분 객체가 독립적으로 존재할 수 있는가
합성: Composition	전체 객체가 부분 객체의 수명을 책임지는가

```
class Player
{
private:
    Weapon* weapon = nullptr; // 연관
};

class Party
{
private:
    Player* members[4] = {}; // 집합
    int memberCount = 0;
};

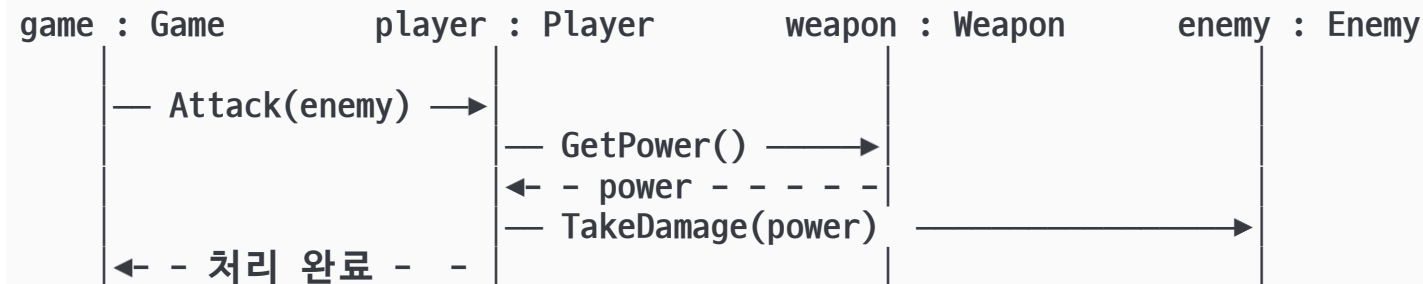
class Character
{
private:
    Vector2 position; // 합성
};
```

UML 관계 기호

- 연관 관계 (Association): – (단순 실선 또는 화살표)
- 집합 관계 (Aggregation): ◇– (속이 빈 마름모 실선)
- 합성 관계 (Composition): ◆– (속이 찬 마름모 실선)



- 클래스 다이어그램은 정적인 구성
- 시퀀스 다이어그램은 객체가 어떤 순서로 협력하는지 표현
- 시간은 위에서 아래로 흐름



요소	의미
참여자	호출에 참여하는 객체
생명선	객체가 존재하는 시간 흐름
메시지	다른 객체의 기능 호출
반환 메시지	호출 결과 반환



3.11.3 시퀀스와 C++ 코드 대응

```
void Player::Attack(Enemy& enemy)
{
    if (weapon == nullptr)
    {
        return;
    }

    int power = weapon->GetPower();
    enemy.TakeDamage(power);
}

int Weapon::GetPower() const
{
    return power;
}

void Enemy::TakeDamage(int damage)
{
    if (damage <= 0)
    {
        return;
    }

    hp -= damage;

    if (hp < 0)
    {
        hp = 0;
    }
}
```

시퀀스 다이어그램	C++ 코드
Attack(enemy)	player.Attack(enemy)
GetPower()	weapon->GetPower()
TakeDamage(power)	enemy.TakeDamage(power)



- UML의 속성은 멤버 변수로 구현
- UML의 연산은 멤버 함수 선언과 정의로 구현
- 상태 변경 규칙은 클래스 내부 함수에서 처리

```
// Character.h
#pragma once
#include <string>
#include "Vector2.h"
class Character
{
public:
    Character(const std::string& name);
    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);
    const std::string& GetName() const;
    const Vector2& GetPosition() const;
    int GetHp() const;

private:
    std::string name;
    Vector2 position;
    int hp = 100
}
```





- 객체지향 모델링: 객체를 식별하고 상태, 책임, 관계와 협력을 설계하는 과정
- UML: 객체의 구조와 동작을 다이어그램으로 표현하는 모델링 언어
- 클래스: 객체의 상태와 기능을 정의하는 C++ 사용자 정의 타입
- class와 struct : 접근 제어 외 동일 설계 의도를 구분하여 사용
- 접근 제어: 객체의 유효한 상태와 캡슐화를 유지하는 데 사용
- 게터와 세터: 기계적으로 추가한다고 좋은 캡슐화가 되는 것은 아님
- this: 현재 멤버 함수를 호출한 객체 자신
- 정적 멤버: 클래스 전체가 공유하는 상태와 기능을 표현
- 전방 선언: 헤더 의존성을 줄이는 데 사용
- UML 관계: C++ 문법뿐 아니라 역할, 소유권, 수명을 함께 고려

